

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 吴竞宸 学号 2024K8009929022 专业 计算机科学与技术
实验项目编号 4.2 实验名称 高速缓存设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

1 总体架构设计

本实验为流水线 RISC-V 处理器设计了独立的指令 Cache 和数据 Cache, 两者均为 8 组 4 路组相联结构, 采用 256 位(8 字)的 Cache 行大小。分离式 Cache 设计(Harvard 架构)消除了取指和访存之间的结构冲突, 允许 IF 级和 MEM 级同时访问各自的 Cache。

指令 Cache(icache)和数据 Cache(dcache)共享相同的基础存储结构: 每组 4 个 way, 每个 way 包含一个 `tag_array` (24 位标签 $\times$ 8 项) 和一个 `data_array` (256 位数据 $\times$ 8 项), 辅以 `valid` 位数组(icache 和 dcache) 和 `dirty` 位数组(仅 dcache)。两种 Cache 的核心差异在于状态机设计: icache 为只读 Cache, 状态机较为简洁; dcache 为可读写 Cache, 需要处理写命中、写回(write-back)和 I/O 旁路(bypass)等复杂情况。

2 指令 Cache(I-Cache)设计

指令 Cache 的状态机包含八个状态: WAIT、TAG_RD、EVICT、MEM_RD、RECV、REFILL、RESP 和 CACHE_RD。

WAIT 状态是空闲状态, 在此状态下 icache 接收 CPU 的取指请求。当握手成功后, 进入 TAG_RD 状态读取标签和有效位进行命中判定。`raddr` 端口为组合逻辑读取, 在 WAIT 状态读取 `cpu_index` 对应的项(提前为 TAG_RD 做准备), 在非 WAIT 状态读取 `req_index` 对应的项。

TAG_RD 状态同时读取 4 个 way 的标签和数据, 并进行命中比较。命中向量 `hit_vector` 的每一位通过 `way_valid[req_index]` 与 `way_tag_dout[i]` 匹配产生。若命中(Read_Hit 为真), 跳转至 CACHE_RD 状态; 若缺失, 跳转至 EVICT 状态。

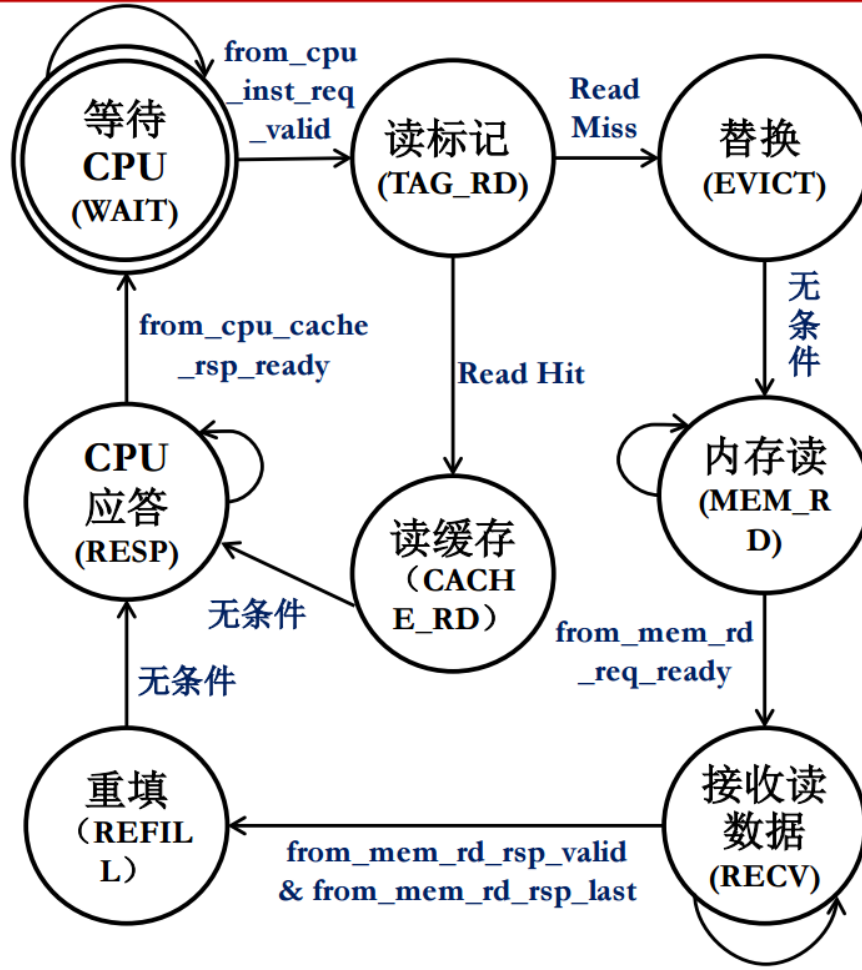


图 1: 指令 Cache 状态转移图

CACHE_RD 状态从命中的 Cache 行中根据 `req_offset[4:2]` 选取对应字, 存入 `resp_data_r` 寄存器, 随后进入 RESP 状态返回数据给 CPU。

EVICT 状态通过轮转计数器 (`rr_counter`) 选出替换路 (`victim_way`), 清除该路对应索引的 `valid` 位。随后进入 MEM_RD 状态向内存发出读请求。

MEM_RD 状态将地址对齐到 32 字节边界并发出读请求。RECV 状态以 8 拍顺序接收从内存返回的 256 位数据, 每拍使用递增的 `recv_cnt` 选择写入 `refill_line_wdata` 的对应字。

REFILL 状态将 `refill_line_wdata` 写入 `victim_way` 的 `data_array` 和 `tag_array`, 同时更新 `valid` 位为有效以及推进轮转计数器。

RESP 状态将 `resp_data_r` 返回给 CPU, 握手完成后回到 WAIT 状态。

指令 Cache 的轮转替换策略为每个组维护一个 2 位的 `rr_counter`, 每次在本组发生替换后对 `counter` 加 1。这是一种简化的 FIFO 近似策略, 实现简单且硬件开销小。由于指令访问通常具有良好的空间局部性, 轮转策略在实际 benchmark 中表现良好。

3 数据 Cache(D-Cache)设计

数据 Cache 的状态机包含十四个状态, 在指令 Cache 的基础上增加了写命中 (`WRITE_HIT`)、写回请求 (`EVICT_WB_REQ`)、写回数据 (`EVICT_WB_DATA`) 以及 I/O 旁路 (`BYP_RD_REQ`、`BYP_RD_RECV`、`BYP_WR_REQ`、`BYP_WR_DATA`) 等状态。

写策略: 本设计采用写回 (write-back) 结合写分配 (write-allocate) 的策略。写命中时, 不清除 `dirty` 位,

数据仅在 Cache 中更新 (WRITE_HIT 状态执行 read-modify-write 操作), 待该行被替换时再统一写回内存。写缺失时, 先将缺失行从内存读入 Cache (MEM_RD→RECV→REFILL), 若替换行为脏行则先写回脏数据 (EVICT_WB_REQ→EVICT_WB_DATA), 在 REFILL 阶段将 CPU 写数据合并到 refill_line_wdata 中。

写命中时的 read-modify-write 逻辑通过字节使能信号 (wstrb) 精确控制: 仅更新 CPU 指定的字节, Cache 行中的其余字节保持不变。合并后的 256 位数据在 WRITE_HIT 状态写入 data_array, 并通过 dirty 位标记该行为脏。

I/O 旁路: 地址空间中小于 0x20 或大于等于 0x40000000 的区域被识别为 I/O 地址, 不走 Cache 通路而是直接通过旁路状态访问内存。读旁路经过 BYP_RD_REQ→BYP_RD_RECV→RESP 路径, 写旁路经过 BYP_WR_REQ→BYP_WR_DATA→WAIT 路径。旁路内存访问为单拍传输 (burst length 为 0), 与 Cache 行填充的 8 拍传输 (burst length 为 7) 区别开来。

替换与写回流程: 当 TAG_RD 状态判定缺失且 victim_dirty 为真时, 进入 EVICT_WB_REQ 状态发起写回请求。写回地址由 victim_tag_r (在 TAG_RD 阶段锁存的替换路标签)、req_index 和 5 位零拼接而成。在 EVICT_WB_DATA 状态, 通过 wb_beat_cnt 控制 8 拍顺序发送 victim_data_line 的每个字到内存写数据通道。写回完成后进入 MEM_RD 状态开始缺失行填充。若 victim_dirty 为假, 缺失路径直接进入 MEM_RD, 跳过写回阶段。

Dirty 位管理: 写命中时将对应 way 和 index 的 dirty 位置 1。REFILL 时, 若当前请求是写操作, dirty 位置 1 (因为写数据已在 REFILL 阶段合并); 若是读操作, dirty 位清零 (从内存读来的行是干净的)。这种精细的 dirty 位管理避免了不必要的写回操作, 减少了内存带宽消耗。

4 存储阵列设计

tag_array 和 data_array 模块分别实现 24 位 × 8 项和 256 位 × 8 项的寄存器阵列。两者均采用单端口 SRAM 风格的接口: waddr、wdata、wen 控制写操作, raddr 控制读操作 (组合逻辑读取), rdata 为读数据输出。

tag_array 模块定义了 TARRAY_DATA_WIDTH=24 和 TARRAY_ADDR_WIDTH=3 的参数, 内部为 8 项的寄存器数组。写使能 (wen) 有效时在时钟上升沿写入, 读操作为组合逻辑输出 (assign rdata = array[raddr])。

data_array 模块与之类似, 但数据宽度为 256 位 (DARRAY_DATA_WIDTH=256)。宽数据通路用于一次读取整个 Cache 行 (8 个 32 位字), 后续在 Cache 控制器中通过 req_offset 选择目标字。

每个 Cache 实例化的 8 个存储模块 (4 个 tag_array + 4 个 data_array) 在 TAG_RD 状态下并行读取, 所有 way 的标签和数据在同一时钟周期内可用。这种并行比较的设计是实现单周期命中判定的关键。

二、 开发成果 (对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 指令 Cache 状态转移逻辑

```
always @(*)
begin
    case(current_state)
        WAIT:
            if(to_cpu_inst_req_ready && from_cpu_inst_req_valid)
                next_state = TAG_RD;
            else next_state = WAIT;
        TAG_RD:
            if(Read_Hit) next_state = CACHE_RD;
            else next_state = EVICT;
        EVICT:
            next_state = MEM_RD;
```

```

MEM_RD:
    if(to_mem_rd_req_valid && from_mem_rd_req_ready)
        next_state = RECV;
    else next_state = MEM_RD;
RECV:
    if(from_mem_rd_rsp_valid && from_mem_rd_rsp_last)
        next_state = REFILL;
    else next_state = RECV;
REFILL: next_state = RESP;
RESP:
    if(to_cpu_cache_rsp_valid && from_cpu_cache_rsp_ready)
        next_state = WAIT;
    else next_state = RESP;
CACHE_RD: next_state = RESP;
default: next_state = WAIT;
endcase
end

```

状态机在设计上保持了简单的线性流程, 大多数状态转移条件来自握手信号。EVICT 状态为组合逻辑直接跳转(无等待条件), 因为轮转替换的 victim 选择和 valid 位清空均可在一个周期内完成。

2 命中判定与数据选取

```

assign hit_vector[0] = way0_valid[req_index] & (way_tag_dout[0] == req_tag);
assign hit_vector[1] = way1_valid[req_index] & (way_tag_dout[1] == req_tag);
assign hit_vector[2] = way2_valid[req_index] & (way_tag_dout[2] == req_tag);
assign hit_vector[3] = way3_valid[req_index] & (way_tag_dout[3] == req_tag);
assign Read_Hit = |hit_vector;

wire [255:0] hit_line_data = (way_data_dout[0] & {256{hit_vector[0]}})
    | (way_data_dout[1] & {256{hit_vector[1]}})
    | (way_data_dout[2] & {256{hit_vector[2]}})
    | (way_data_dout[3] & {256{hit_vector[3]}});

```

命中检测采用全并行比较方式: 4 个 way 的标签与请求标签同时比较, 4 个比较器在单周期内产生 hit_vector。数据选择采用掩码或(masked-OR)方式, 利用 hit_vector 的每 bit 作为 256 位宽的掩码, 将命中的 way 数据选出, 未命中的 way 贡献全零。这种方式避免了多路选择器的级联延迟。

3 轮转替换策略

```

reg [1:0] rr_counter [7:0];
wire [1:0] victim_way = rr_counter[req_index];

always @(posedge clk) begin
    if (rst) begin
        for (idx = 0; idx < 8; idx = idx + 1)
            rr_counter[idx] <= 2'b0;
    end else if (is_refill)

```

```

rr_counter[req_index] <= rr_counter[req_index] + 1'b1;
end

```

每个 Cache 组维护独立的 2 位轮转计数器，初始为 0。每次在 REFILL 状态（本组发生了替换），对应组的计数器自增。该策略保证同一组的 4 个 way 按循环顺序被替换，实现简单且不需要额外的访问历史信息。对于 4 路组相联结构，2 位计数器足以表达 4 个 way 的轮转顺序（0→1→2→3→0...）。

4 数据 Cache 写命中和替换逻辑

写命中操作的核心是 read-modify-write: 读出命中行，按字节使能合并 CPU 写数据，再写回数组。

```

wire [255:0] cache_wdata = (write_hit_data & {256{is_write_hit}})
    | (refill_merged & {256{is_refill && req_is_write_r}})
    | (refill_line_wdata & {256{is_refill && !req_is_write_r}});

assign data_wen[0] = (is_refill && (victim_way_r == 2'd0))
    | (is_write_hit && (hit_way_r == 2'd0));
// ... way 1, 2, 3

```

data_wen 信号在写命中和 REFILL 两种情况下都会使能。写命中时，根据 TAG_RD 阶段锁存的 hit_way_r 选择正确的 way 进行写入。REFILL 时，根据锁存的 victim_way_r 写入。cache_wdata 的来源也根据不同状态进行多路选择：写命中时使用合并了 CPU 写数据的 write_hit_data，REFILL 写缺失时使用合并了写数据的 refill_merged，REFILL 读缺失时使用纯 refill_line_wdata。

替换过程包括脏行写回和新行填充两个阶段。写回地址由被替换行的标签、索引和 5 位零拼接而成：

```

assign to_mem_wr_req_addr = ({victim_tag_r, req_index, 5'b0}
    & {32{is_evict_wb_req}})
    | ({req_addr_r[31:2], 2'b0}
    & {32{is_byp_wr_req}});

```

写回数据通过 8 拍的突发传输发送，每拍由 wb_beat_cnt 选择 victim_data_line 的对应字。写回完成（wb_beat_cnt 到达 7 且最后一拍握手成功）后进入 MEM_RD 状态，开始缺失行的填充流程。

三、 学习总结（总结设计、开发与调试过程中遇到的问题，及实验中的收获与不足）

1 问题总结（实验过程中遇到的问题、对问题的思考过程及解决方法）

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	RTL 语法	icache	报错	多个时序逻辑(always 块)对同一个寄存器进行操作	合并

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

类别	名称	体会	后续计划
亮点	状态机设计	状态序列清晰	可考虑引入非阻塞 Cache
不足	替换策略	轮转替换可能存在颠簸	研究 pseudo-LRU 替换策略
不足	验证方法	波形分析效率较低	学习使用性能计数器辅助调试

表 2: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

• 如何实现大于 4 byte 的指令/数据交互?

对于 32-bit 位宽的内存访问接口,一次 Cache Block 数据交互就是突发读/写 8 个数据。以本设计为例,Cache 行大小为 256 位(8 个 32 位字),当发生 Cache miss 需要从内存读取整个 Block 时,通过突发传输(burst)一次发送 8 拍数据。发送端(Cache 或内存)在每拍传输一个 32 位数据,连续 8 拍即可完成 256 位 Block 的全部传输。burst length 参数用于标记传输的拍数——8 拍传输对应的 len 值用最后一个数据的下标值来标记,即 len=7。接收端通过计数器 recv_cnt 跟踪当前拍的位置,将每拍数据写入 refill_line_wdata 的对应字位置。

这个接口同样适用于大于单个 Block 的数据交互:只需将数据切分为多个 Block,每个 Block 走一次 8 拍突发传输即可。

• 如果用上述接口只传输 32-bit 数据(len = 0),last 在哪个位置拉高?

当 len=0 时,表示本次突发传输仅包含 1 拍数据(即单拍传输,并非 0 拍)。在这种模式下,last 信号在唯一的数据拍(即第一拍)同时拉高。也就是说,发送端在第一拍数据有效时将 last 信号置为 1,接收端在收到该拍数据并检测到 last 为高后,即知道本次传输已完成。

本设计的 I/O 旁路(bypass)访问正是使用 len=0 的单拍传输模式——读旁路在 BYP_RD_RECV 状态等待单拍数据返回,last 与 valid 同时为高时即完成接收,无需 recv_cnt 计数。而 Cache 行填充(缺失处理)使用 len=7 的 8 拍传输,last 在第 8 拍拉高。

五、 实验时长

在课后,你花费了大约 5 小时完成此次实验,其中各个阶段的用时如表 3 所示。

步骤	时长(小时)	说明
设计	0.5	Cache 架构设计与状态机规划
开发	3	I-Cache 替换策略、写回逻辑实现
开发	1	D-Cache 状态机、替换策略、写回逻辑实现
调试	0.5	写回时序与 dirty 位管理的协同调试

表 3: 实验时长